



Stage和Screen

北京理工大学计算机学院
金旭亮

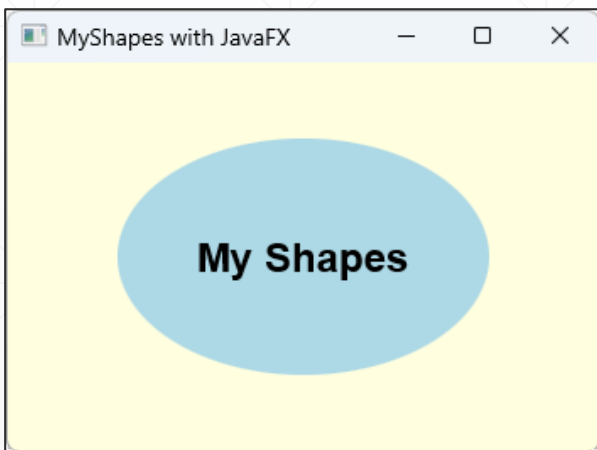
概述



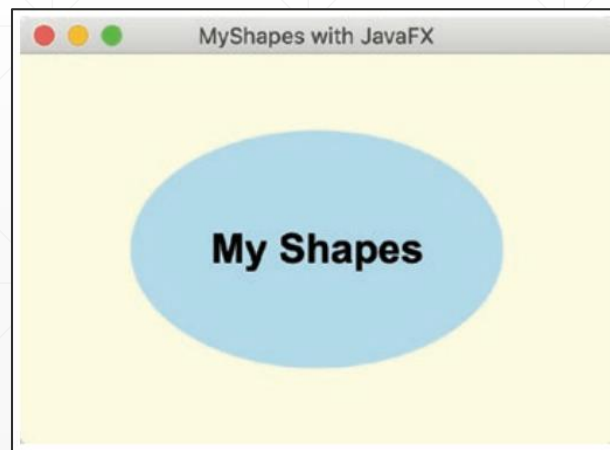
JavaFX使用Stage抽象出GUI程序的顶层容器，在PC上，这个顶层容器就是“窗体（Window）”。



Stage本质上是对底层操作系统的Window的封装，直接使用了底层操作系统的Window外观与交互特性（比如拥有符合操作系统外观特性的最大最小按钮等）。窗体中显示的具体内容，则由JavaFX接管，由JavaFX应用负责定义，JavaFX引擎负责渲染和生成，在交互特性上具有跨平台的一致性。



Windows下的Stage



Mac下的Stage

Screen对象



窗体 (Stage) 在显示器屏幕 (Screen) 上显示，其左上角坐标 (x, y) 和宽度 (width)、高度 (height) 决定了其显示区域。



JavaFX使用Screen来封装本机显示器信息，通过它可以查询到各显示器的分辨率。



一台计算机，可能拥有多个显示器，相应地，对应多个Screen对象。

使用Screen对象获取显示器信息

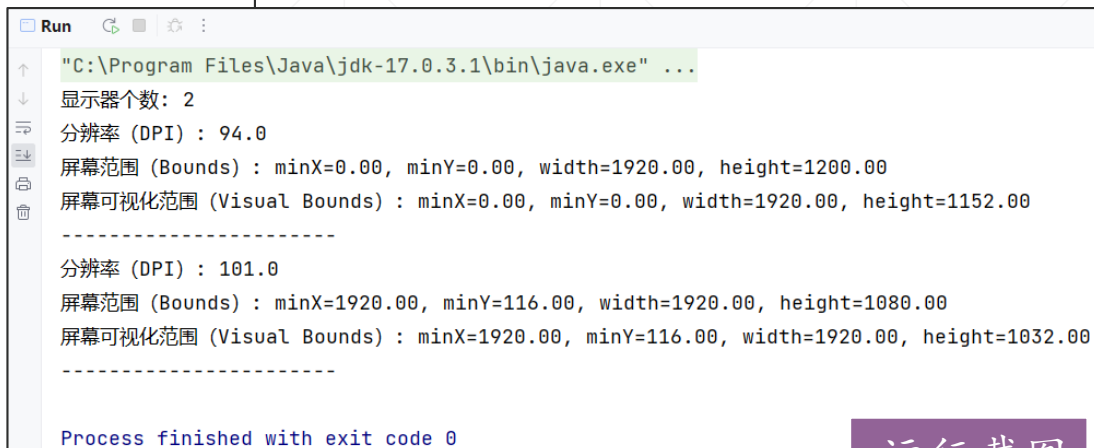
ScreenDetailsApp.java

```
public void print(Screen s) {  
    System.out.println("分辨率 (DPI) : " + s.getDpi());  
    System.out.print("屏幕范围 (Bounds) : ");  
    Rectangle2D bounds = s.getBounds();  
    print(bounds);  
    System.out.print("屏幕可视化范围 (Visual Bounds) : ");  
    Rectangle2D visualBounds = s.getVisualBounds();  
    print(visualBounds);  
    System.out.println("-----");  
}
```

2 usages

```
public void print(Rectangle2D r) {  
    System.out.format("minX=%.2f, minY=%.2f, width=%.2f, height=%.2f\n",  
        r.getMinX(), r.getMinY(),  
        r.getWidth(), r.getHeight());  
}
```

```
@Override  
public void start(Stage stage) throws Exception {  
    ObservableList<Screen> screenList = Screen.getScreens();  
    System.out.println("Screens Count: " + screenList.size());  
    for (Screen screen : screenList) {  
        print(screen);  
    }  
    Platform.exit();  
}
```



```
Run  
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" ...  
显示器个数: 2  
分辨率 (DPI) : 94.0  
屏幕范围 (Bounds) : minX=0.00, minY=0.00, width=1920.00, height=1200.00  
屏幕可视化范围 (Visual Bounds) : minX=0.00, minY=0.00, width=1920.00, height=1152.00  
-----  
分辨率 (DPI) : 101.0  
屏幕范围 (Bounds) : minX=1920.00, minY=116.00, width=1920.00, height=1080.00  
屏幕可视化范围 (Visual Bounds) : minX=1920.00, minY=116.00, width=1920.00, height=1032.00  
-----  
Process finished with exit code 0
```

运行截图

如何实现全屏显示?



FullScreen.java

```
@Override
public void start(Stage primaryStage) throws Exception {
    VBox vBox = new VBox();
    vBox.setSpacing(10);
    vBox.setAlignment(Pos.CENTER);
    vBox.setPadding(new Insets(topRightBottomLeft: 10));
    vBox.getChildren().add(new Text("ESC退出全屏模式"));
    Scene scene = new Scene(vBox, width: 600, height: 400);

    primaryStage.setScene(scene);
    primaryStage.setFullScreen(true);
    primaryStage.show();
}
```

设为全屏显示时，会忽略掉这里指定的宽度和高度，隐藏窗体（Stage）标题栏，采用当前Screen的尺寸显示窗体。



压ESC可退出全屏.....

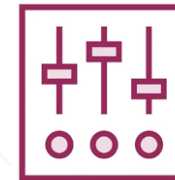
小技巧：如何禁止窗体改变大小



非全屏显示情况下，如果不希望用户使用鼠标调整Stage的大小，可以调用Stage的以下方法：

```
stage.setResizable(false);
```


设定窗体透明度



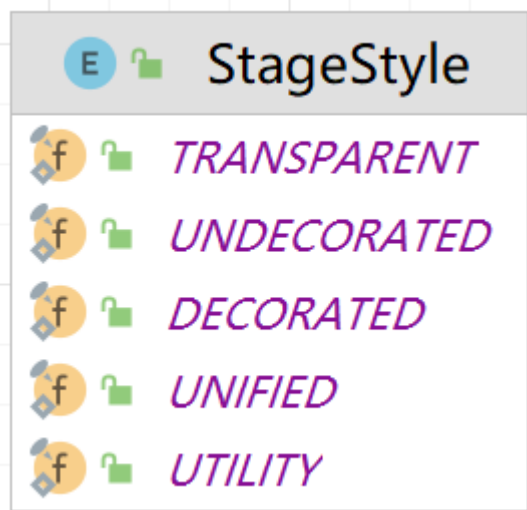
可以调用stage的setOpacity()方法设定窗体的透明度，其值为 $[0, 1.0]$ 之间的double数值。



StageOpacity.java

设定窗体风格

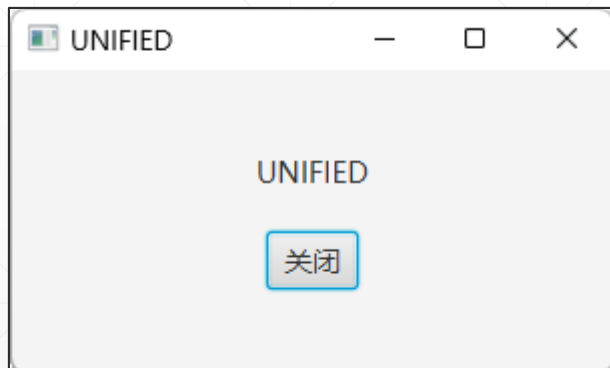
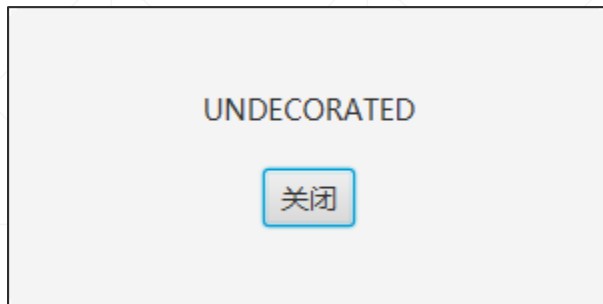
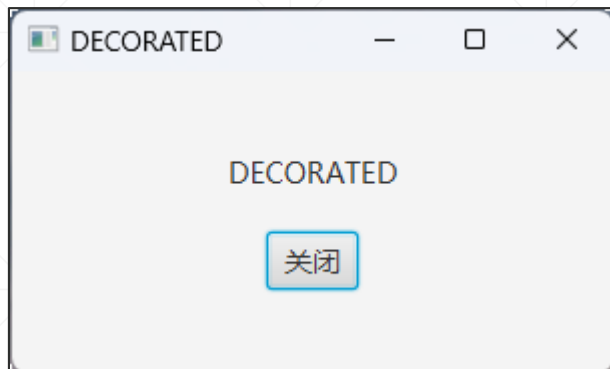
调用Stage.initStyle()方法
设定其Style。



```
//按照特定的样式显示窗体
1 usage
private void show(Stage stage, Label styleLabel, StageStyle style) {
    //显示样式名称
    styleLabel.setText(style.toString());
    stage.setTitle(style.toString());
    //设置窗体样式
    stage.initStyle(style);
    // 对于透明样式，需要将窗体背景设置为null，否则，会得到一个白色的背景
    if (style == TRANSPARENT) {
        stage.getScene().setFill(null);
        stage.getScene().getRoot().setStyle(
            "-fx-background-color: transparent");
    } else if(style == UNIFIED) {
        stage.getScene().setFill(Color.TRANSPARENT);
    }
    // 显示窗体
    stage.show();
}
```

StageStyleApp.java

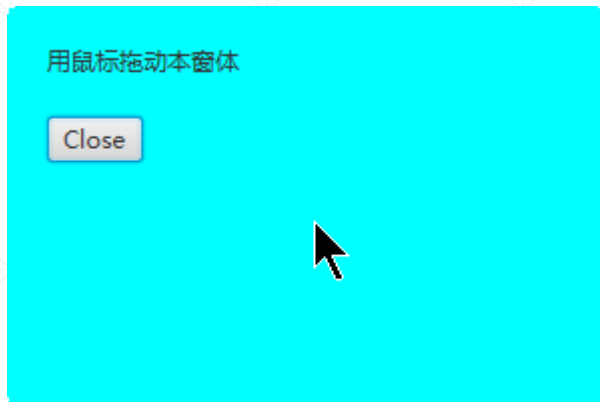
不同样式的窗体 (windows 11下截图)



不同样式的窗体，在不同的操作系统上可能有不同的交互特性，需要进行测试才能确定。

StageStyleApp.java

应用实例：拖动无边框窗体



DraggingStage.java

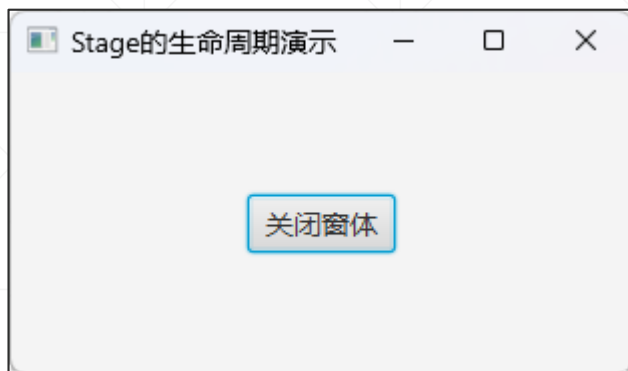
要点：

- (1) 窗体样式选择UNDECORATED
- (2) 在Stage的MouseDown事件 中，计算x和y方向拖动的位移（距离）
- (3) 在Stage的MouseDragged事件中，通过设置Stage的X和Y坐标实现窗体移动。

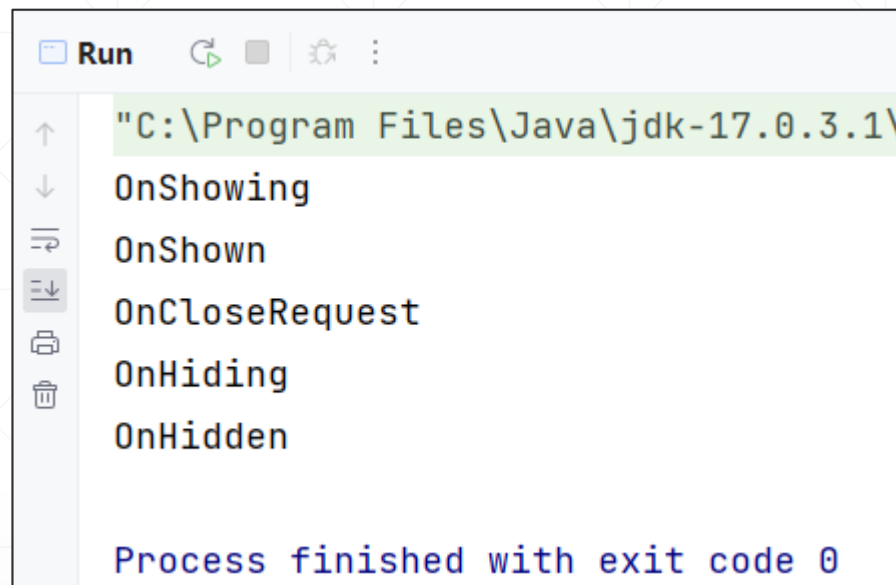


请对照上述要点自行阅读示例源码。

Stage生命周期事件



StageLifecycle.java



在窗体从显示到关闭的过程中，Stage会触发特定的事件，我们可以编写代码响应这些事件。

示例分析-1

通过显式关闭窗体，
触发相应事件

在此方法中响应窗
体触发的各种事件

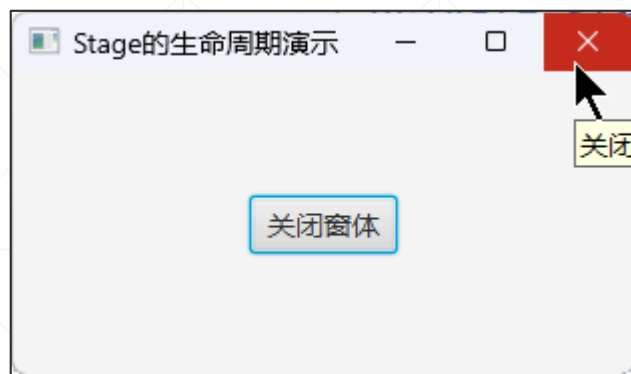
```
@Override
public void start(Stage primaryStage) throws Exception {

    Button button = new Button(s: "关闭窗体");
    button.setOnAction(e -> primaryStage.close());
    VBox root = new VBox();
    root.setPadding(new Insets( topRightBottomLeft: 10));
    root.setAlignment(Pos.CENTER);
    root.getChildren().add(button);
    Scene scene = new Scene(root, width: 310, height: 150);
    primaryStage.setTitle("Stage的生命周期演示");
    primaryStage.setScene(scene);

    responseToStageLifecycle(primaryStage);

    primaryStage.show();
}
```

示例分析-2



触发

点击关闭图标，Stage会触发onCloseRequest事件，需要时，可以“吃掉”它，仅允许程序调用Stage.close()方法显式关闭窗口体。

```
private static void responseToStageLifecycle(Stage primaryStage) {
    primaryStage.setOnCloseRequest(e -> {
        System.out.println("OnCloseRequest");
        e.consume(); //禁止用户点击窗体“关闭”图标关闭自己
    });
    primaryStage.setOnHidden(e -> {
        System.out.println("OnHidden");
    });
    primaryStage.setOnHiding(e -> {
        System.out.println("OnHiding");
    });

    primaryStage.setOnShowing(e -> {
        System.out.println("OnShowing");
    });

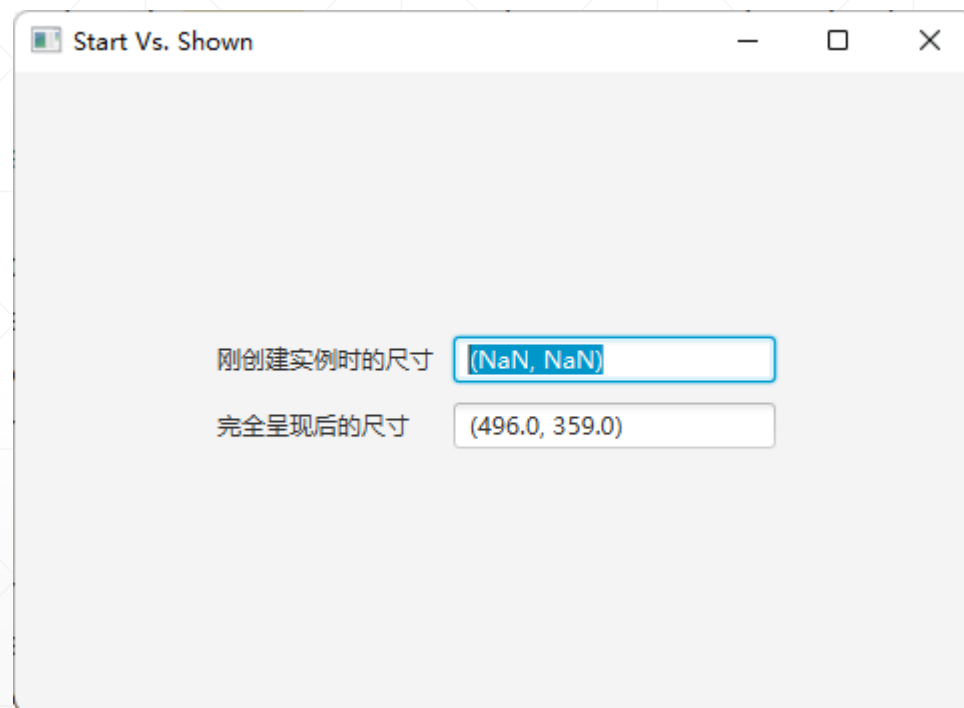
    primaryStage.setOnShown(e -> {
        System.out.println("OnShown");
    });
}
```

何时可获取控件尺寸数据?

```
// 在show()之前, 尺寸信息不可获得
startX.set( primaryStage.getWidth() );
startY.set( primaryStage.getHeight() );

//窗体完全呈现, 尺寸信息现在可以获得
primaryStage.setOnShown( (evt) -> {
    shownX.set( primaryStage.getWidth() );
    shownY.set( primaryStage.getHeight() );
});

primaryStage.setTitle("Start Vs. Shown");
primaryStage.show();
```



StartVsShownJavaFXApp.java

应用实例：在屏幕中央显示Stage

每一个窗体，在显示前后都可以设定它的左上角坐标 (x,y) 和显示的宽度 (width) 与高度 (height)，但只有等到显示之后，才能获取到它的真实尺寸属性值，因此以下方法，要在stage显示之后调用才生效。

```
private static void moveStageToCenter(Stage stage) {  
    //获取当前主屏幕的尺寸信息  
    Rectangle2D bounds = Screen.getPrimary().getVisualBounds();  
    //只有在窗体显示之后，才能获取它的尺寸  
    double x = bounds.getMinX() + (bounds.getWidth() - stage.getWidth()) / 2.0;  
    double y = bounds.getMinY() + (bounds.getHeight() - stage.getHeight()) / 2.0;  
    //设置窗体的左上角坐标  
    stage.setX(x);  
    stage.setY(y);  
}
```

显示之前，Stage的宽度和高度信息，都是无法获取的。

CenterScreen.java



想显示多个窗体，就创建多个Stage实例，并调用它们的show()方法即可。

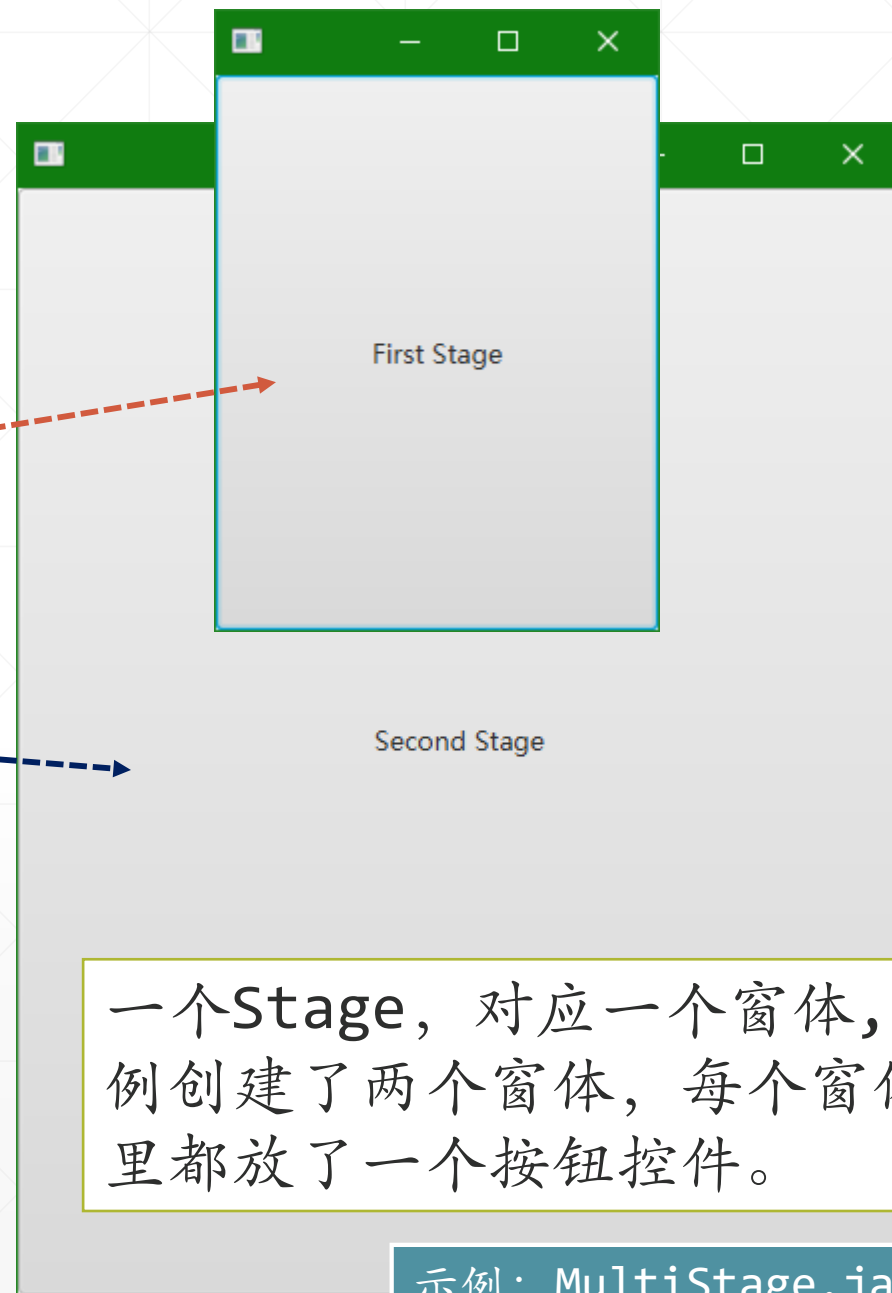
Stage实例

show()



“多舞台”示例

```
public class MultiStage extends Application {  
    @Override  
    public void start(Stage primaryStage) throws Exception {  
  
        Button btnFirst = new Button(s: "First Stage");  
        Scene scene = new Scene(btnFirst, width: 200, height: 250);  
        primaryStage.setScene(scene);  
        primaryStage.show();  
  
        Stage otherStage = new Stage();  
        Button btnSecond = new Button(s: "Second Stage");  
        Scene scene2 = new Scene(btnSecond, width: 400, height: 500);  
        otherStage.setScene(scene2);  
        //如果指定otherStage的所有者是primaryStage, 则当Owner关闭时  
        //otherStage也会关闭  
        otherStage.initOwner(primaryStage);  
        otherStage.show();  
    }  
  
    public static void main(String[] args) { launch(args); }  
}
```



示例：MultiStage.java

窗体之间的“所有者”关系



Stage有一个owner属性，用于指定它的“拥有者”。

```
Button btn = new Button(s: "创建子窗体");
btn.setOnAction(e -> {
    Stage newStage = new Stage();
    newStage.setTitle("子窗体");
    //设定所有者关系
    newStage.initOwner(primaryStage);
    newStage.setWidth(300);
    newStage.setHeight(200);
    newStage.show();
});
```

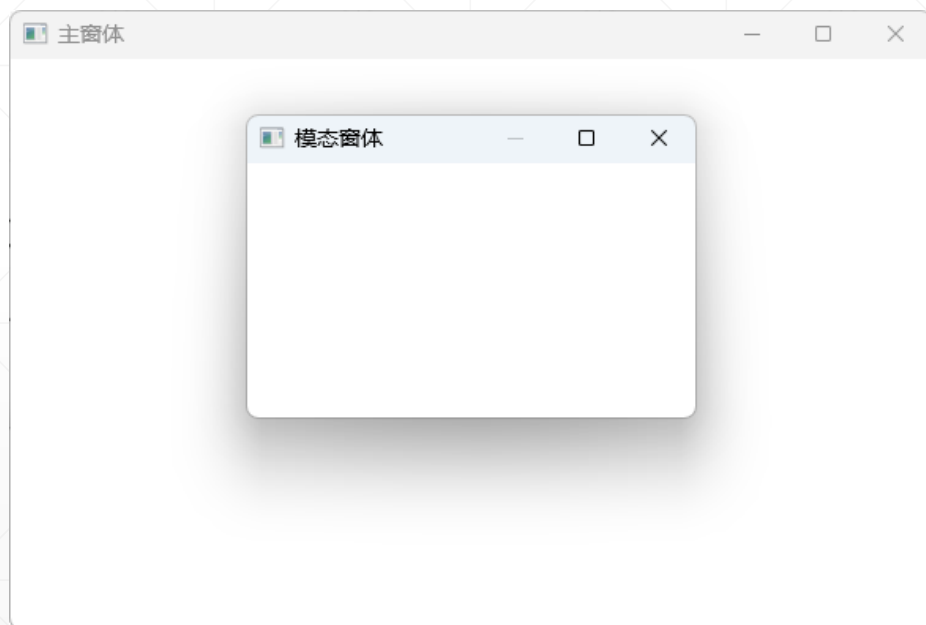
StageOwner.java

当这么干了之后，primaryStage最小化（或隐藏）时，它所拥有的所有“子窗体”，也会随之最小化或隐藏）。

primaryStage关闭（close）后，它所拥有的所有“子窗体”，也会随之关闭。

模态窗体

JavaFX定义了三种Stage的显示方式：
NONE、WINDOW_MODAL和
APPLICATION_MODAL。



小窗体不关闭，大窗体将无法访问。

ModalStage.java

```
@Override
public void start(Stage primaryStage) {
    primaryStage.setTitle("主窗体");
    primaryStage.setWidth(600);
    primaryStage.setHeight(400);
    primaryStage.show();

    Stage stage = new Stage();
    stage.setTitle("模态窗体");
    stage.setWidth(300);
    stage.setHeight(200);
    //以模态方式显示
    stage.initModality(Modality.APPLICATION_MODAL);
    stage.show();
}
```

Modality.APPLICATION_MODAL：表明它将阻塞本JavaFX应用的其他窗体。

Modality.WINDOW_MODAL：表明它将阻塞本窗体的“所有者（owner）窗体”，本应用的其他窗体不受影响。

show() vs. showAndWait()



要显示一个Stage，可以在new出一个Stage()对象之后，调用它的show()或showAndWait()方法。



这两个方法的共同之处在于：两者都会显示一个Stage并启动一个独立的消息循环。

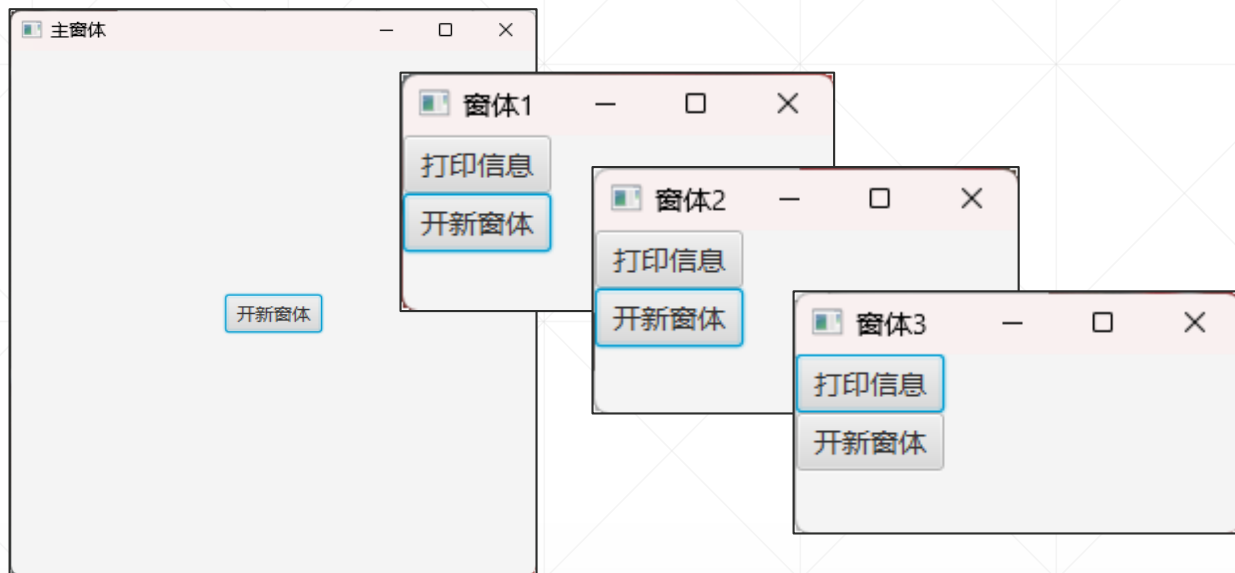


这两个方法的区别在于：show()在显示完Stage之后，会立即返回，而showAndWait()则会阻塞等待显示的Stage关闭或隐藏之后才返回。



不能对主窗体（即JavaFX应用启动时创建的第一个窗体）调用showAndWait()方法。

ShowAndWaitApp.java示例

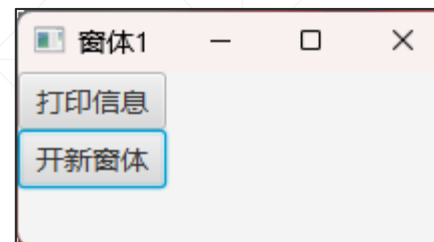


```
Run
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
在stage.showAndWait()之前, 窗体 1
在stage.showAndWait()之前, 窗体 2
在stage.showAndWait()之前, 窗体 3
打印的信息来自于窗体 #1
打印的信息来自于窗体 #2
打印的信息来自于窗体 #3
在stage.showAndWait()之后, 窗体 3
在stage.showAndWait()之后, 窗体 2
在stage.showAndWait()之后, 窗体 1
Process finished with exit code 0
```

在本讲示例运行后，主窗体创建窗体1，窗体1创建窗体2，窗体2创建窗体3，.....，只要你是按照这个顺序创建的三个窗体，那么，你可以随意点击“打印信息”按钮输出信息，也可以以随意的顺序关闭这三个窗体，这时你会发现，只有等待三个窗体全部关闭之后，那些“在...之后”的信息，才会出现。

示例关键代码

```
private void open(int stageNumber) {  
    Stage stage = new Stage();  
    stage.setTitle("窗体" + stageNumber);  
  
    Button sayHelloButton = new Button("打印信息");  
    sayHelloButton.setOnAction(  
        e -> System.out.println(  
            "打印的信息来自于窗体 #" + stageNumber));  
  
    Button openButton = new Button("开新窗体");  
    openButton.setOnAction(e -> open(++counter));  
  
    ...  
  
    System.out.println("在stage.showAndWait()之前, 窗体 " +  
        stageNumber);  
    stage.showAndWait();  
    System.out.println("在stage.showAndWait()之后, 窗体 " +  
        stageNumber);  
}
```



点击“开新窗体”按钮，会创建一个Stage对象，并且调用它的showAndWait()方法显示在屏幕上。

这一句，在窗体关闭之前，不会运行